###Part 1: Exponential Smoothing Methods

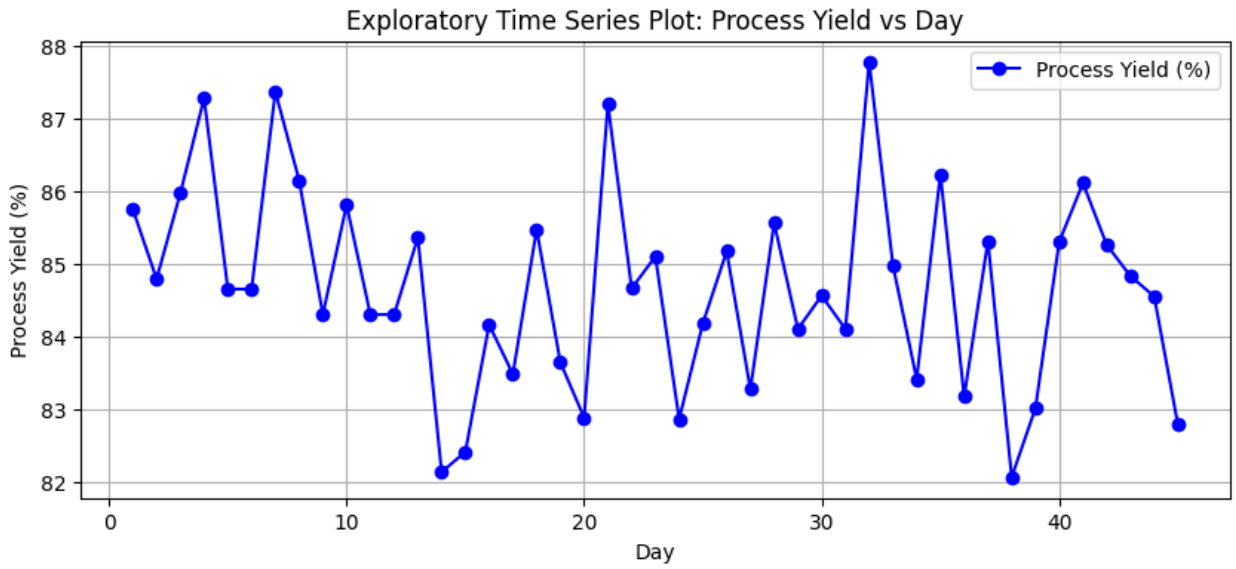
Item 1.1: Simple Exponential Smoothing (SES) for Constant Processes

A. Exploratory Time Series Plot

```
import pandas as pd
import matplotlib.pyplot as plt

df_ses = pd.read_csv('/content/sample_data/ses-process-yield.csv')

# Process Yield vs Day
plt.figure(figsize=(10, 4))
plt.plot(df_ses['Day'], df_ses['Process_Yield'], marker='o',
linestyle='-', color='b', label='Process Yield (%)')
plt.title("Exploratory Time Series Plot: Process Yield vs Day")
plt.xlabel("Day")
plt.ylabel("Process Yield (%)")
plt.grid(True)
plt.legend()
plt.show()
```



The series fluctuates around a stable mean (~85%) without any long-term trend or seasonality. Therefore the series is STATIONARY in mean and variance, making SES appropriate.

B. SES Implementation

```
import numpy as np
import pandas as pd

df_ses = pd.read_csv('/content/sample_data/ses-process-yield.csv')
y_ses = df_ses['Process_Yield'].values
```

```
N_ses = len(y_ses)

def run_ses(y, alpha):
    F = np.zeros(len(y) + 1)
    F[1] = y[0]
    for t in range(2, len(y) + 1):
        F[t] = alpha * y[t-1] + (1 - alpha) * F[t-1]
    return F

# 1-step-ahead forecasts for Days 2 through 45
F_020 = run_ses(y_ses, 0.20)
F_040 = run_ses(y_ses, 0.40)

# Display
ses_results_df = pd.DataFrame({
    'Day': df_ses['Day'],
    'Actual_Yield': y_ses,
    'Forecast_alpha_0.20': F_020[:N_ses],
    'Forecast_alpha_0.40': F_040[:N_ses]
})

print("1-Step-Ahead Forecasts (Days 2 through 45):")
print(ses_results_df.head(5))
```

```
1-Step-Ahead Forecasts (Days 2 through 45):
   Day  Actual_Yield  Forecast_alpha_0.20  Forecast_alpha_0.40
0    1         85.75              0.00000              0.00000
1    2         84.79             85.75000             85.75000
2    3         85.97             85.55800             85.36600
3    4         87.28             85.64040             85.60760
4    5         84.65             85.96832             86.27656
```

C. Accuracy Comparison

```
actuals = y_ses[1:]
f_020 = F_020[1:N_ses]
f_040 = F_040[1:N_ses]

# MSE and MAPE
mse_020 = np.mean((actuals - f_020) ** 2)
mape_020 = np.mean(np.abs((actuals - f_020) / actuals)) * 100

mse_040 = np.mean((actuals - f_040) ** 2)
mape_040 = np.mean(np.abs((actuals - f_040) / actuals)) * 100

print(f"Alpha = 0.20 -> MSE: {mse_020:.4f}, MAPE: {mape_020:.4f}%")
print(f"Alpha = 0.40 -> MSE: {mse_040:.4f}, MAPE: {mape_040:.4f}%")
```

```
Alpha = 0.20 -> MSE: 2.0943, MAPE: 1.3597%
Alpha = 0.40 -> MSE: 2.3135, MAPE: 1.4558%
```

The alpha = 0.20 yields superior in-sample fitting accuracy.

D. Out-of-Sample Forecasting

```
alpha = 0.20
F = np.zeros(N_ses + 1)
F[1] = y_ses[0]

for t in range(2, N_ses + 1):
    F[t] = alpha * y_ses[t-1] + (1 - alpha) * F[t-1]

forecast_day_46 = F[N_ses]

print(f"1-step-ahead point forecast for Day 46: {forecast_day_46:.4f}
%")
```

```
1-step-ahead point forecast for Day 46: 84.3351%
```

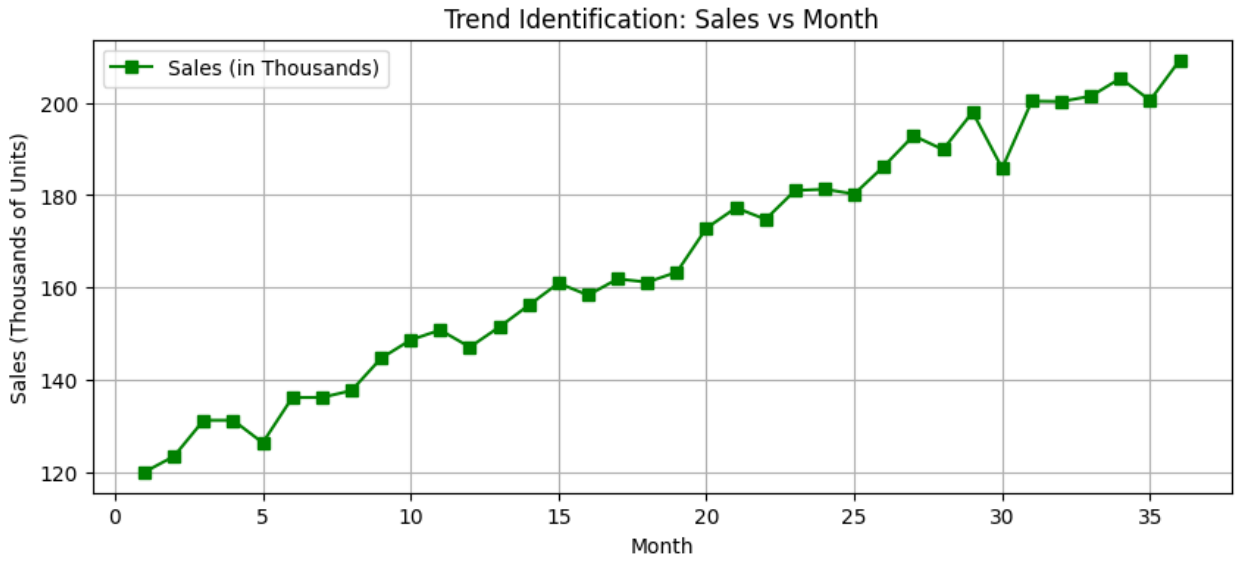
Item 1.2: Holt's Linear Exponential Smoothing for Trended Data

A. Trend Identification

```
import pandas as pd
import matplotlib.pyplot as plt

df_holts = pd.read_csv('/content/sample_data/holts-medical-sales.csv')

# Plot Sales vs Month
plt.figure(figsize=(10, 4))
plt.plot(df_holts['Month'], df_holts['Sales_Thousands'], marker='s',
linestyle='-', color='g', label='Sales (in Thousands)')
plt.title("Trend Identification: Sales vs Month")
plt.xlabel("Month")
plt.ylabel("Sales (Thousands of Units)")
plt.grid(True)
plt.legend()
plt.show()
```



SES is inadequate because it assumes a constant mean and lacks a trend parameter. On trended data, SES constantly lags behind actuals. Thats why Holt's method is required as Holt's Linear Smoothing includes two parameters (alpha for Level, beta for Trend) to explicitly capture and project trended patterns.

B. Holt's Implementation

```
df_holts = pd.read_csv('/content/sample_data/holts-medical-sales.csv')
y_holts = df_holts['Sales_Thousands'].values
N_holts = len(y_holts)

alpha_h, beta_h = 0.30, 0.20

L = np.zeros(N_holts + 1)
T = np.zeros(N_holts + 1)
F = np.zeros(N_holts + 1)

L[1] = y_holts[0]
T[1] = y_holts[1] - y_holts[0]

# Recursively update L_t and T_t for Months 2 through 36
for t in range(2, N_holts + 1):
    F[t] = L[t-1] + T[t-1]
    L[t] = alpha_h * y_holts[t-1] + (1 - alpha_h) * (L[t-1] + T[t-1])
    T[t] = beta_h * (L[t] - L[t-1]) + (1 - beta_h) * T[t-1]

# Results table
holts_results_df = pd.DataFrame({
    'Month': df_holts['Month'],
    'Actual_Sales': y_holts,
    'Level_Lt': L[1:N_holts+1],
    'Trend_Tt': T[1:N_holts+1],
    'Forecast_Ft': F[1:N_holts+1]
```

```
})

print("Holt's Smoothing Values (Months 1 through 36):")
print(holts_results_df.tail(10))

print(f"\nFinal State at Month 36:")
print(f"Level L_36 = {L[36]:.4f}")
print(f"Trend T_36 = {T[36]:.4f}")
```

```
Holt's Smoothing Values (Months 1 through 36):
    Month  Actual_Sales    Level_Lt  Trend_Tt  Forecast_Ft
26     27        192.88  189.722324  2.822555   188.369034
27     28        189.87  191.742415  2.662062   192.544878
28     29        197.98  195.477134  2.876593   194.404476
29     30        185.83  194.596609  2.125170   198.353727
30     31        200.38  197.819245  2.344663   196.721778
31     32        200.30  200.204735  2.352828   200.163908
32     33        201.45  202.225295  2.286375   202.557564
33     34        205.32  204.754169  2.334874   204.511669
34     35        200.54  205.124330  1.941932   207.089043
35     36        209.23  207.715383  2.071756   207.066262

Final State at Month 36:
Level L_36 = 207.7154
Trend T_36 = 2.0718
```

C. Multi-Step Out-of-Sample Forecasts

```
alpha_h, beta_h = 0.30, 0.20

L = np.zeros(N_holts + 1)
T = np.zeros(N_holts + 1)

# Initialization
L[1] = y_holts[0]
T[1] = y_holts[1] - y_holts[0]

for t in range(2, N_holts + 1):
    L[t] = alpha_h * y_holts[t-1] + (1 - alpha_h) * (L[t-1] + T[t-1])
    T[t] = beta_h * (L[t] - L[t-1]) + (1 - beta_h) * T[t-1]

forecast_month_37 = L[36] + 1 * T[36]
forecast_month_38 = L[36] + 2 * T[36]

print(f"Point Forecast for Month 37 (tau = 1): {forecast_month_37:.4f}
thousand units")
print(f"Point Forecast for Month 38 (tau = 2): {forecast_month_38:.4f}
thousand units")
```

```
Point Forecast for Month 37 (tau = 1): 209.7871 thousand units
Point Forecast for Month 38 (tau = 2): 211.8589 thousand units
```

###**Part 2: ARIMA Models & Diagnostic Checking**

Item 2.1: Stationary ARIMA Model Identification & Fitting

A. Data Exploration & ACF/PACF Generation

```
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf

df_server = pd.read_csv('/content/sample_data/arima-server-load.csv')
y_server = df_server['Memory_Load_Pct']

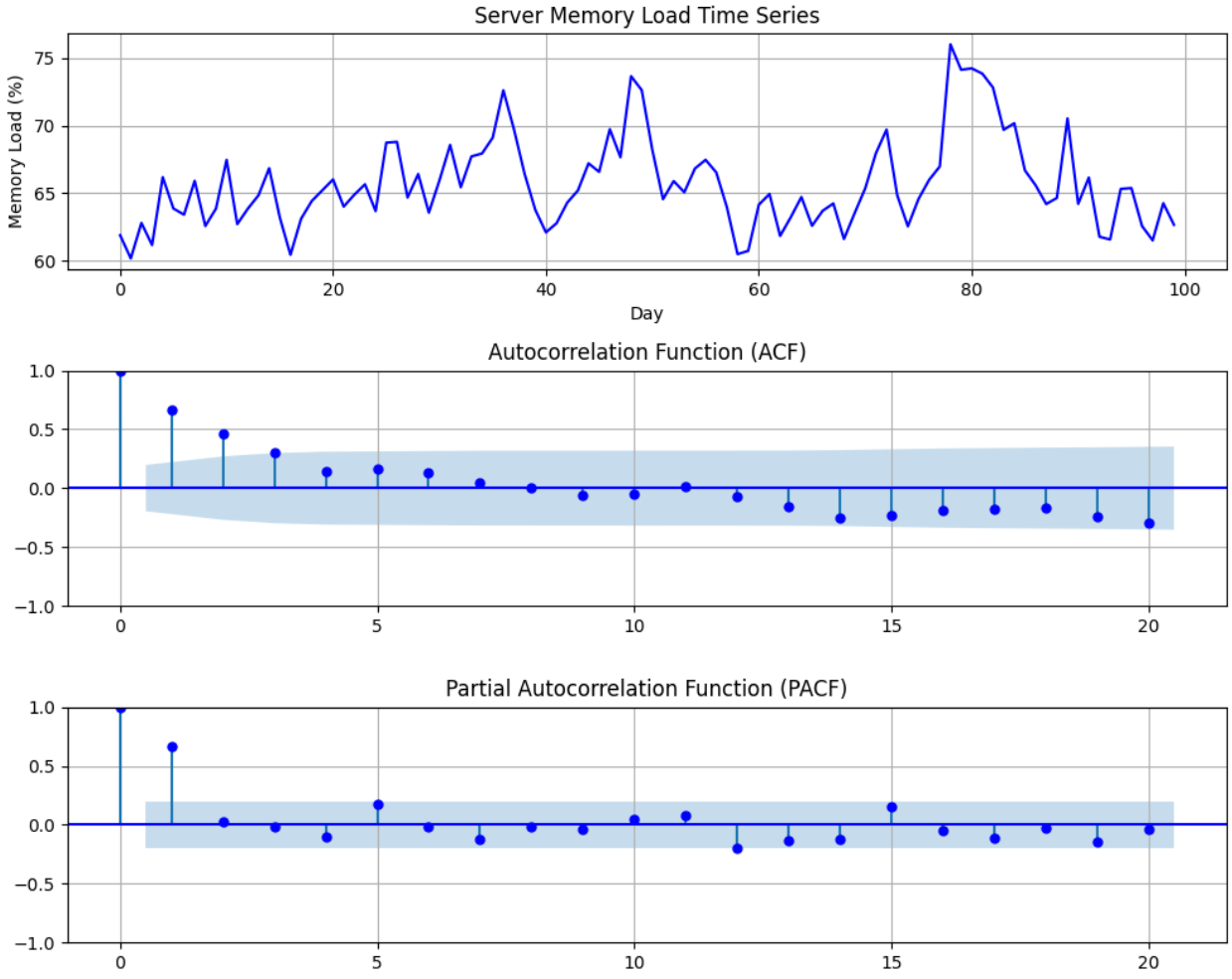
fig, axes = plt.subplots(3, 1, figsize=(10, 8))

# Line plot
axes[0].plot(y_server, color='blue', linewidth=1.5)
axes[0].set_title("Server Memory Load Time Series")
axes[0].set_xlabel("Day")
axes[0].set_ylabel("Memory Load (%)")
axes[0].grid(True)

# ACF plot (k = 20)
plot_acf(y_server, lags=20, ax=axes[1], color='blue')
axes[1].set_title("Autocorrelation Function (ACF)")
axes[1].grid(True)

# PACF plot (k = 20)
plot_pacf(y_server, lags=20, ax=axes[2], color='blue')
axes[2].set_title("Partial Autocorrelation Function (PACF)")
axes[2].grid(True)

plt.tight_layout()
plt.show()
```



B. Model Identification

1. Stationary (d = 0)
   - The server memory load time series fluctuates around a stable mean level(approx. 65%) without a permanent upward or downward trend. The sample ACF decays steadily and drops inside the 95% confidence bounds after lag 3. Therefore, no differencing is required (d = 0).
2. Autoregressive Order (p = 1)
   - The sample PACF shows a single significant spike at lag 1 (≈0.67) and immediately cuts off after lag 1, staying within the shaded 95% significance band for all higher lags. A sharp cutotat lag 1 in the PACF is the classic signature of an AR(1) process.
3. Moving Average (q = 0)
   - The sample ACF exhibits a smooth, exponential decays starting from lag 1 ($\approx 0.67) down through lag 3 before dying out. Exponential decays in the ACF paired with a sharp cutoff in the PACF confirms there is no moving average component (q = 0).

Tentative Model Specification:

- ARIMA (1,0,0) or AR(1)

C. Parameter Estimation

```
from statsmodels.tsa.arima.model import ARIMA

# Fit ARIMA model
p, d, q = 1, 0, 0
model_21 = ARIMA(y_server, order=(p, d, q))
res_21 = model_21.fit()

print("--- Estimated Parameters ---")
print(res_21.summary())
```

```
--- Estimated Parameters ---
                               SARIMAX Results

==============================================================================
========
Dep. Variable:        Memory_Load_Pct   No. Observations:
100
Model:                 ARIMA(1, 0, 0)   Log Likelihood
-232.634
Date:                Mon, 05 Oct 2026   AIC
471.268
Time:                        08:16:29   BIC
479.083
Sample:                             0   HQIC
474.431
                                - 100

Covariance Type:                  opg

==============================================================================
========
                 coef    std err          z      P>|z|      [0.025
0.975]
------------------------------------------------------------------------------
--------
const         65.5628      0.813     80.633      0.000      63.969
67.156
ar.L1          0.6734      0.089      7.571      0.000       0.499
0.848
sigma2         6.1029      0.731      8.354      0.000       4.671
7.535
==============================================================================
=============
Ljung-Box (L1) (Q):                   0.06   Jarque-Bera (JB):
12.22
Prob(Q):                              0.81   Prob(JB):
0.00
```

```
Heteroskedasticity (H):                    1.55   Skew:
0.62
Prob(H) (two-sided):                       0.22   Kurtosis:
4.17
================================================================================
=============

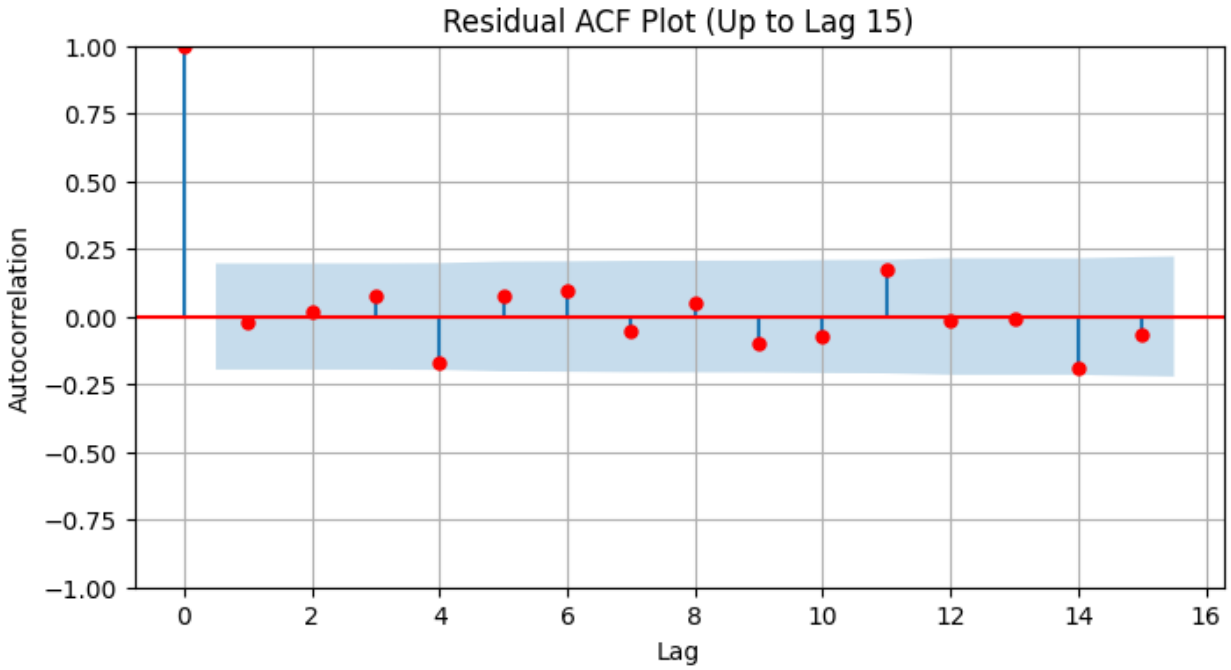
Warnings:
[1] Covariance matrix calculated using the outer product of gradients
(complex-step).
```

D. Diagnostic Checking

```
import matplotlib.pyplot as plt
from statsmodels.graphics.tsaplots import plot_acf

# Extract model residuals
residuals_21 = res_21.resid

fig, ax = plt.subplots(figsize=(8, 4))
plot_acf(residuals_21, lags=15, ax=ax, color='red')
ax.set_title("Residual ACF Plot (Up to Lag 15)")
ax.set_xlabel("Lag")
ax.set_ylabel("Autocorrelation")
ax.grid(True)
plt.show()
```



The autocorrelation spikes fall within the shaded 95% confidence region, therefore the residuals behave as WHITE NOISE, confirming adequate model fit.

E. Forecasting

```
forecast_21 = res_21.get_forecast(steps=2)
fc_21_summary = forecast_21.summary_frame(alpha=0.05)

print(fc_21_summary[['mean', 'mean_ci_lower', 'mean_ci_upper']])
```

```
Memory_Load_Pct       mean  mean_ci_lower  mean_ci_upper
100              63.608147      58.766253      68.450041
101              64.246594      58.409313      70.083875
```

Item 2.2: Non-Stationary ARIMA Model Identification & Differencing

A. Stationarity Assessment & Raw ACF/PACF

```
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf

df_power = pd.read_csv('/content/sample_data/arima-power-demand.csv')
y_power = df_power['Power_Demand_MW']

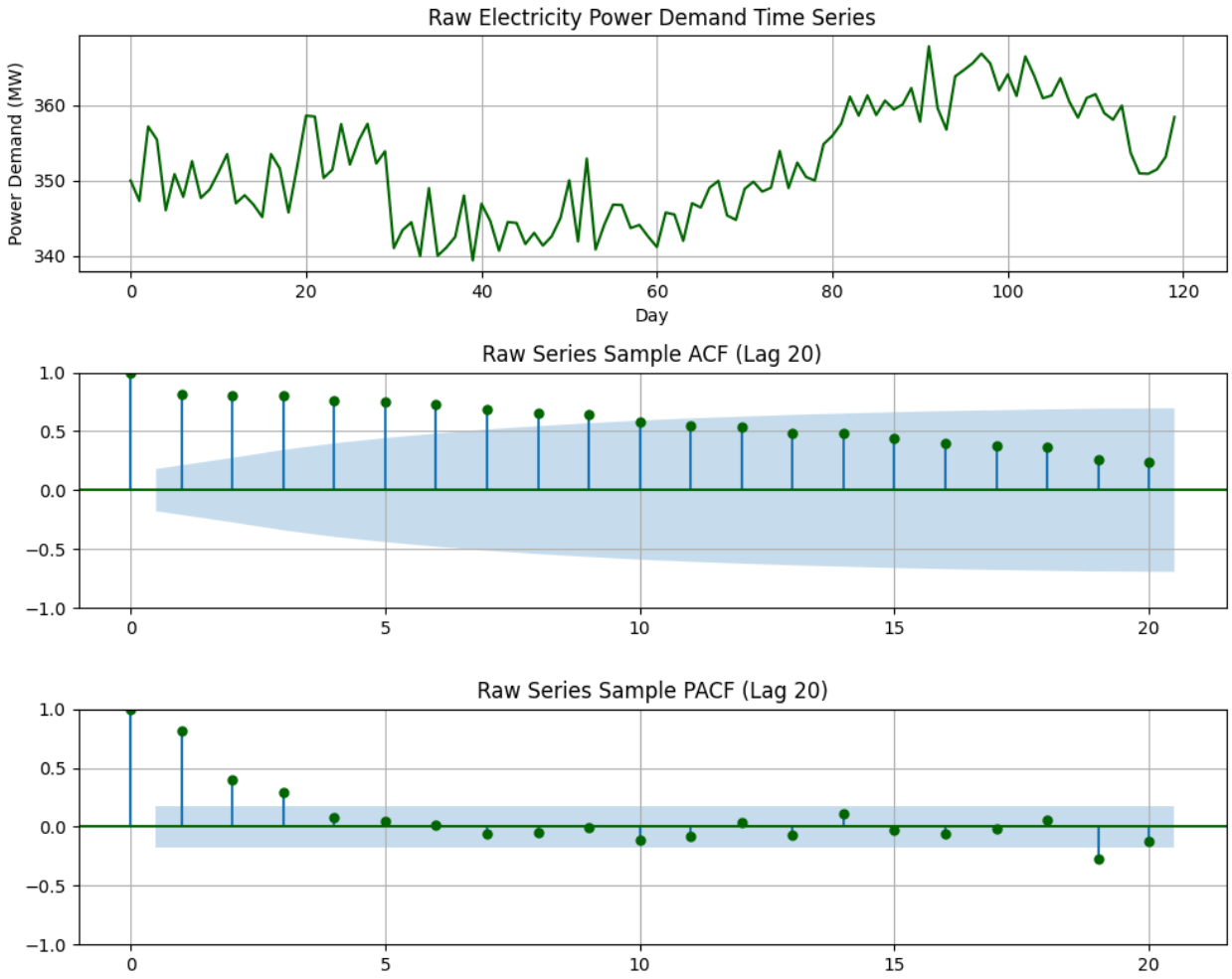
fig, axes = plt.subplots(3, 1, figsize=(10, 8))

# Raw line plot
axes[0].plot(y_power, color='darkgreen', linewidth=1.5)
axes[0].set_title("Raw Electricity Power Demand Time Series")
axes[0].set_xlabel("Day")
axes[0].set_ylabel("Power Demand (MW)")
axes[0].grid(True)

# Raw ACF
plot_acf(y_power, lags=20, ax=axes[1], color='darkgreen')
axes[1].set_title("Raw Series Sample ACF (Lag 20)")
axes[1].grid(True)

# Raw PACF
plot_pacf(y_power, lags=20, ax=axes[2], color='darkgreen')
axes[2].set_title("Raw Series Sample PACF (Lag 20)")
axes[2].grid(True)

plt.tight_layout()
plt.show()
```



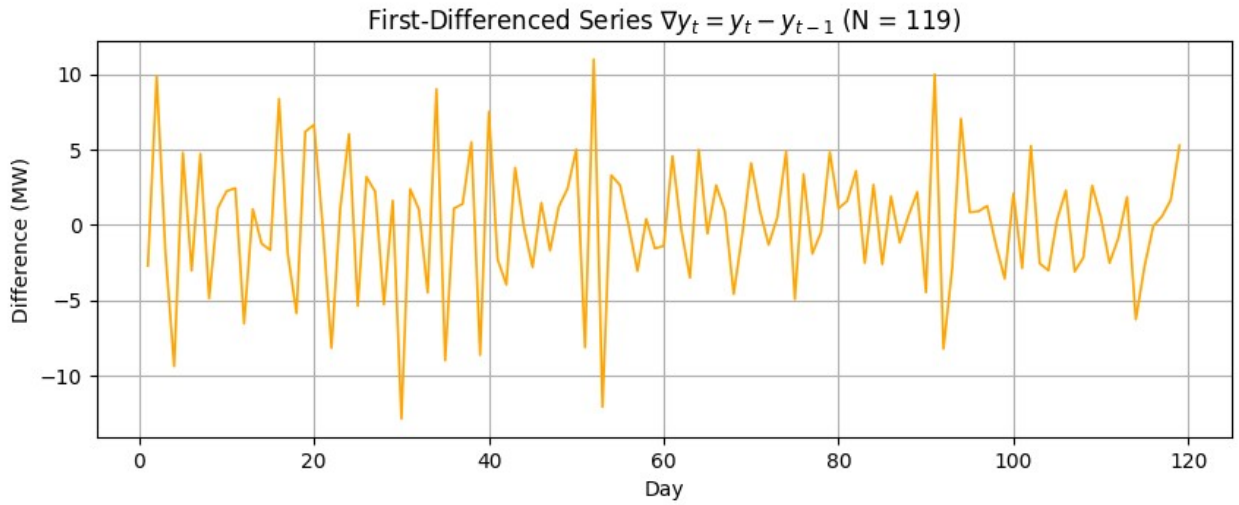
The extremely slow linear decay in the raw ACF confirms that the series is non-stationary. While the mean is time-dependent (trending), requiring differencing (d = 1) to achieve stationarity.

B. First-Differencing

```
import matplotlib.pyplot as plt

diff_power = y_power.diff().dropna()

# Plot first-differenced time series
plt.figure(figsize=(10, 3.5))
plt.plot(diff_power, color='orange', linewidth=1.2)
plt.title(r"First-Differenced Series $\nabla y_t = y_t - y_{t-1}$ (N =
119)")
plt.xlabel("Day")
plt.ylabel("Difference (MW)")
plt.grid(True)
plt.show()
```



C. Differenced ACF/PACF Analysis & Model Order Identification

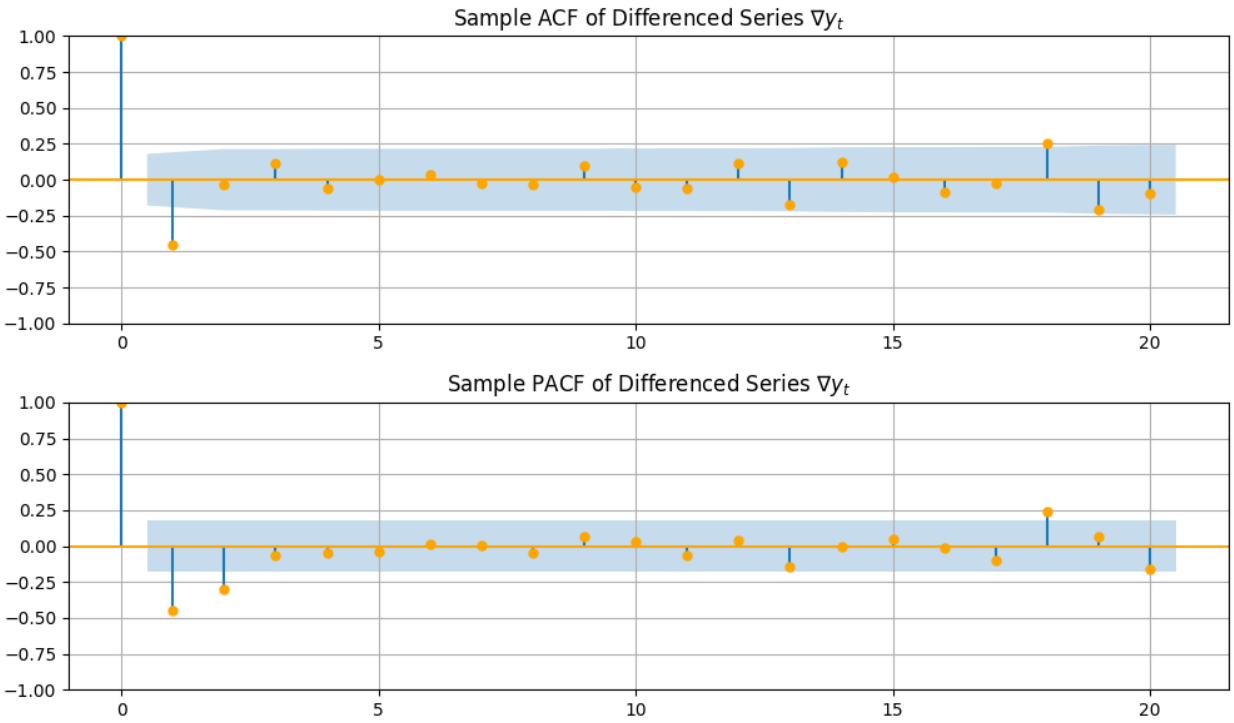
```
import matplotlib.pyplot as plt
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf

# Plot ACF and PACF for differenced series
fig, axes = plt.subplots(2, 1, figsize=(10, 6))

plot_acf(diff_power, lags=20, ax=axes[0], color='orange')
axes[0].set_title(r"Sample ACF of Differenced Series $\nabla y_t$")
axes[0].grid(True)

plot_pacf(diff_power, lags=20, ax=axes[1], color='orange')
axes[1].set_title(r"Sample PACF of Differenced Series $\nabla y_t$")
axes[1].grid(True)

plt.tight_layout()
plt.show()
```



1. Differencing order (d = 1)
   - First-differencing successfully remove the non-stationary trend, making the differenced series fluctuate around zero.
2. Moving average order (q = 1)
   - The ACF of ∇y_t shows a single significant negative spike at lag 1 (≈ -0.45) that clearly cuts through the 95% confidence band. All higher lags ($k \geq 2$) fall inside the shaded significance bounds. A sharp cutoff at lag 1 in the ACF indicates q = 1.
3. Autoregressive order (p = 0 or p = 1)
   - The PACF of ∇y_t shows exponential/oscillating decays across early lags. A sharp cutoff in the ACF paired with decaying PACF confirms a primary Moving Average process (MA(1)) on the difference data.

Tentative Model Specifications:

- ARIMA (0,1,1) or ARIMA (1,1,1)

D. Parameter Estimation

```
from statsmodels.tsa.arima.model import ARIMA

p, d, q = 1, 1, 1
model_22 = ARIMA(y_power, order=(p, d, q))
res_22 = model_22.fit()

# Print model summary
print("--- Parameter Estimation Summary ---")
print(res_22.summary())
```

```
--- Parameter Estimation Summary ---
                               SARIMAX Results

==========================================================================
========
Dep. Variable:         Power_Demand_MW   No. Observations:
120
Model:                  ARIMA(1, 1, 1)   Log Likelihood
-326.532
Date:                 Mon, 05 Oct 2026   AIC
659.064
Time:                         08:38:29   BIC
667.401
Sample:                             0   HQIC
662.450
                                - 120

Covariance Type:                   opg

==========================================================================
========
                 coef    std err          z      P>|z|      [0.025
0.975]
--------------------------------------------------------------------------
--------
ar.L1         -0.0927      0.150     -0.617      0.538      -0.387
0.202
ma.L1         -0.5343      0.128     -4.179      0.000      -0.785
-0.284
sigma2        14.1024      1.714      8.227      0.000      10.743
17.462
==========================================================================
=============
Ljung-Box (L1) (Q):                   0.01   Jarque-Bera (JB):
0.74
Prob(Q):                              0.93   Prob(JB):
0.69
Heteroskedasticity (H):               0.50   Skew:
-0.11
Prob(H) (two-sided):                  0.03   Kurtosis:
3.31
==========================================================================
=============

Warnings:
[1] Covariance matrix calculated using the outer product of gradients
(complex-step).
```

E. Diagnostic Checking & Forecasting

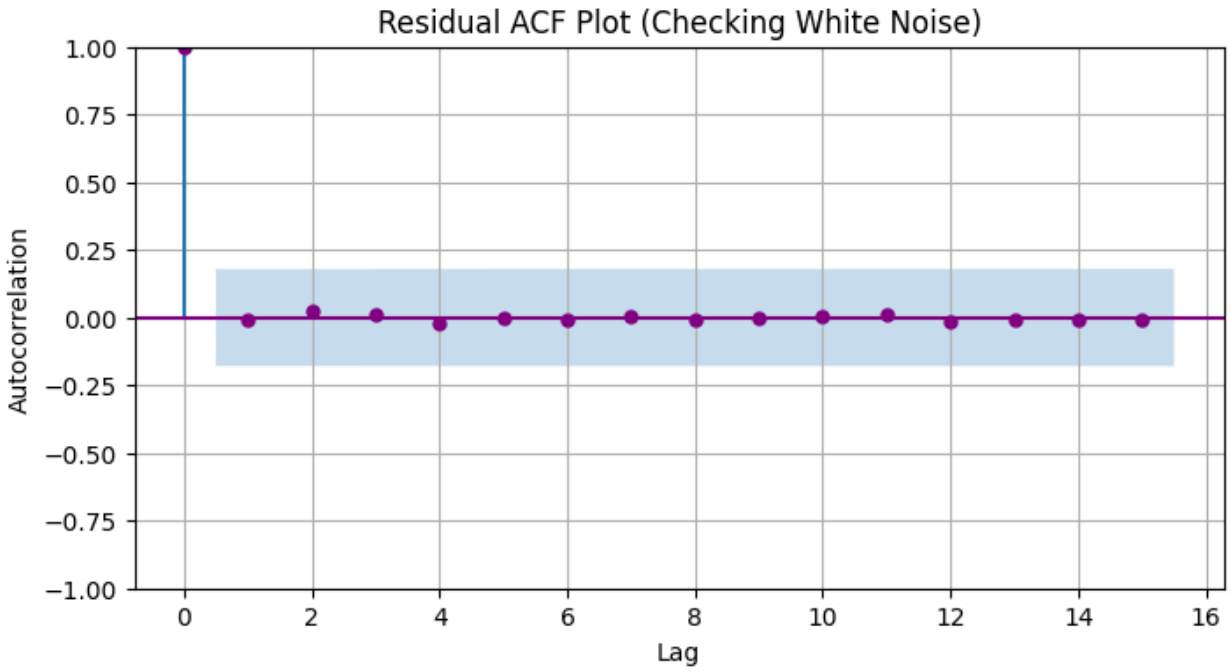
```python
import matplotlib.pyplot as plt
from statsmodels.graphics.tsaplots import plot_acf

residuals_22 = res_22.resid

fig, ax = plt.subplots(figsize=(8, 4))
plot_acf(residuals_22, lags=15, ax=ax, color='purple')
ax.set_title("Residual ACF Plot (Checking White Noise)")
ax.set_xlabel("Lag")
ax.set_ylabel("Autocorrelation")
ax.grid(True)
plt.show()

forecast_22 = res_22.get_forecast(steps=2)
fc_22_summary = forecast_22.summary_frame(alpha=0.05)

print(fc_22_summary[['mean', 'mean_ci_lower', 'mean_ci_upper']])
```




```
Power_Demand_MW         mean  mean_ci_lower  mean_ci_upper
120              354.760499     347.400213     362.120785
121              355.100540     347.244763     362.956316
```